



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México, Escuela de Ingeniería y Ciencias.

Unidad de formación:

TC3009C.601

Inteligencia artificial avanzada para la ciencia de datos.

**Módulo 2 Implementación de una solución empleando un framework o librería
de aprendizaje automático**

Equipo 5

Emiliano Caballero Mendoza A01749050

Índice

1. [Repositorio de GitHub](#)
 2. [Implementación del Algoritmo](#)
 3. [Pruebas y Predicciones](#)
 4. [Entorno de Ejecución](#)
 5. [Resultados y Análisis de Desempeño](#)
 - 5.1 [Datasets de Entrenamiento y Prueba](#)
 - 5.2 [Matriz de Confusión y Métricas](#)
 - 5.3 [Análisis y Conclusión](#)
 6. [Correcciones y Retroalimentación](#)
-

1. Repositorio de GitHub

- URL del Repositorio: <https://github.com/Wolv92/perceptron-training>

2. Implementación del Algoritmo

Descripción del algoritmo seleccionado y detalles sobre su programación manual.

- **Algoritmo implementado:** DBSCAN
- **Descripción técnica:** DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) es un algoritmo de aprendizaje no supervisado que agrupa datos con base en la densidad de puntos en el espacio de características, identificando regiones densas (núcleos) y expandiendo los clusters a partir de ellas. A diferencia de K-means, DBSCAN no requiere especificar de antemano el número de clusters, y es capaz de identificar valores atípicos, etiquetándolos como puntos de ruido (etiqueta **-1**) en lugar de forzarlos a pertenecer a algún grupo.

El algoritmo se basa en dos hiperparámetros principales:

- **eps:** distancia máxima entre dos puntos para que sean considerados vecinos.
- **min_samples:** número mínimo de puntos vecinos (dentro del radio **eps**) que debe tener un punto para ser considerado un *core point*, es decir, el núcleo a partir del cual se expande un cluster.

(DBSCAN — Scikit-Learn 1.9.0 Documentation, n.d.)

Para la programación de este algoritmo se utilizó el framework **scikit-learn**, mediante la clase **DBSCAN**. La selección de los hiperparámetros no fue arbitraria: se empleó la técnica del gráfico de k-distancia, mediante la clase **NearestNeighbors**, calculando la distancia de cada punto a su k-ésimo vecino más cercano (**con k=5**) y ordenando estos valores de menor a mayor.

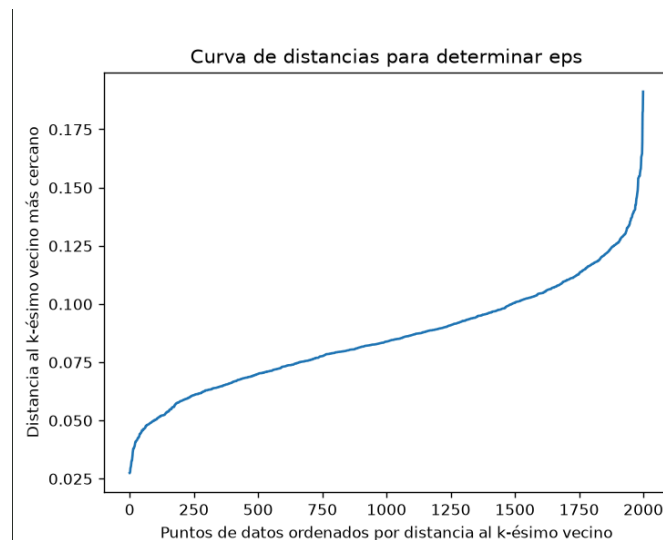


Figura 1. Gráfica de curva de distancias para determinar eps

El punto de inflexión ("codo") de esta curva se utilizó como estimación del valor óptimo de **eps (0.12)**, mientras que **min_samples** se fijó en 5, consistente con el valor de **k** empleado en el gráfico.

3. Pruebas y Predicciones

- **Set de datos utilizado: Mobile Price Classification** (*Mobile Price Classification*, n.d.)
- **Método de visualización:** Consola y gráficas
- **Ejemplos de predicciones:**

Para el modelo, las variables utilizadas de nuestro set de datos fueron **battery_power** y **ram**, ambas normalizadas antes de ser utilizadas dentro del algoritmo.

Para este algoritmo, se asigna cada punto a un cluster descubierto a partir de la densidad de los datos; el objetivo es que el algoritmo pueda determinar, dependiendo de esa densidad, a qué cluster asignar cada punto. La variable

objetivo original del dataset, **price_range**, presenta 4 categorías, utilizadas únicamente como referencia comparativa:

0	económico
1	intermedio
2	elevado
3	muy elevado

Tabla 1. Categorías de price_range

A partir de la densidad de los datos, el algoritmo asigna cada punto a un cluster, o lo marca como ruido (etiqueta **-1**) si no pertenece a ninguna región suficientemente densa. A continuación se muestran 9 ejemplos representativos, incluyendo el valor original de **price_range** de cada celular, a fin de comparar. Es importante destacar que la columna **price_range** no fue utilizada por el algoritmo de clustering.

battery_power	ram	Cluster asignado	price_range real
842	2549	0	1
1021	2631	0	2
563	2603	0	2
615	2769	0	2
1821	1411	0	1
1859	1067	0	1
1821	3220	0	3
1954	700	0	0
1445	1099	0	0

Tabla 2. Ejemplo de predicciones

Se observa que todos los celulares de esta muestra fueron asignados al cluster 0, el cual concentra aproximadamente el 98% de los registros del dataset, mientras que los clusters 1, 2 y 3 corresponden a agrupaciones

considerablemente más pequeñas, y 23 puntos fueron identificados como ruido.

4. Entorno de Ejecución

Confirmación de la portabilidad del código.

- **Lenguaje de programación:** Python
- **Archivo principal:** db-scan.py
- **Instrucciones de compilación/ejecución:** Ir a la carpeta db-scan-training desde tu consola, una vez estando en la carpeta correr el siguiente comando.
py db-scan.[py](#)

```
\db-scan-training> py db-scan.py
```

5. Resultados y Análisis de Desempeño

5.1 Datasets de Entrenamiento y Prueba

Debido a DBScan es un algoritmo de aprendizaje no supervisado, no es necesario una división entre conjuntos de entrenamiento y conjuntos de prueba, el algoritmo analiza la densidad de los datos completos que se proporciona, si no un proceso de generalización de datos no vistos en el sentido tradicional.

- **Dataset utilizado: train.csv** (Mobile Price Classification, Kaggle), utilizado en su totalidad (2000 registros).
- **Variables utilizadas: battery_power y ram**, normalizadas antes de aplicar el algoritmo.
- **Variable de referencia (no utilizada por el algoritmo): price_range**, empleada únicamente con fines comparativos para evaluar si los clusters encontrados coinciden con las categorías reales de precio.

5.2 Evaluación del Clustering

Se utilizó una **tabla de contingencia** para comparar los clusters encontrados contra las categorías reales de **price_range** (no utilizadas durante el clustering), junto con dos métricas específicas de evaluación de clustering: **Silhouette Score** y **Adjusted Rand Index (ARI)**.

Visualización de los clusters:

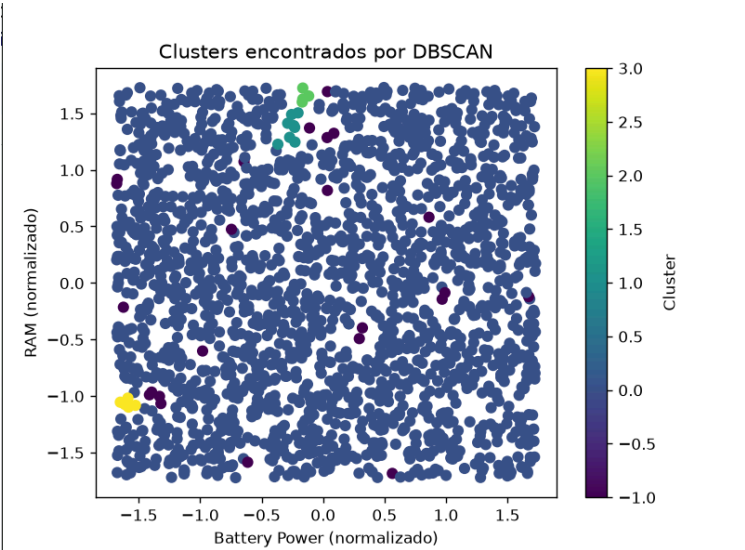


Figura 2. Clusters encontrados por DBSCAN (battery_power vs ram, normalizados)

Tabla de contingencia (**clusters encontrados vs. price_range real**):

Cluster	0 (económico)	1 (intermedio)	2 (elevado)	3 (muy elevado)
-1 (ruido)	6	5	6	6
0	490	494	494	482
1	0	0	0	8
2	0	0	0	4
3	4	1	0	0

Tabla 3. Tabla de contingencia: clusters vs. categorías reales de precio

Métricas de evaluación:

Métrica	Valor Obtenido
Clusters encontrados	4
Puntos de ruido	23
Silhouette Score	-0.2221
Adjusted Rand Index (ARI)	0.0001

Tabla 4. Métricas de evaluación de clustering

5.3 Análisis y Conclusión

El desempeño del modelo, en términos de calidad de clustering, fue deficiente. El Silhouette Score obtenido (-0.2221) indica que, en promedio, los puntos están más cerca de otros clusters distintos al que fueron asignados, lo cual sugiere una asignación geoméricamente inconsistente. Por su parte, el Adjusted Rand Index (0.0001) muestra que la partición encontrada por DBSCAN no guarda ninguna relación significativa con las categorías reales de **price_range**; un valor tan cercano a cero es prácticamente equivalente a una asignación aleatoria. La tabla de contingencia refuerza esta conclusión: el cluster 0, que concentra el 98% de los datos, contiene las 4 categorías de precio distribuidas de forma casi idéntica entre sí, evidenciando que no hay capacidad discriminativa real en la agrupación obtenida.

Es importante notar que esto no significa que las variables seleccionadas (**battery_power y ram**) carezcan de relación con el precio del celular. Es posible que la relación entre estas variables y el precio sea de tipo aproximadamente lineal o continuo, pero sin formar agrupaciones densas y claramente delimitadas geoméricamente, que es lo único que DBSCAN es capaz de detectar.

La selección de los hiperparámetros resultó determinante y, a su vez, reveló una fragilidad importante en el modelo. Se experimentó con distintos valores de **eps**, obteniendo resultados muy dispares: con **eps=0.057** se generaron 77 clusters (excesivamente fragmentados), con **eps=0.10** se obtuvieron 36, con **eps=0.12** se logró la configuración deseada de 4 clusters (23 puntos de ruido), y con un incremento mínimo a **eps=0.1315** el resultado colapsó a un único cluster. Esta alta

sensibilidad ante variaciones pequeñas del parámetro sugiere que la solución de 4 clusters no corresponde a una estructura de densidad robusta en los datos, sino a un punto de transición particularmente inestable entre la fragmentación excesiva y la fusión total.

Una posible explicación adicional es que dos variables no sean suficientes para capturar la estructura de densidad necesaria para diferenciar categorías de precio; el dataset cuenta con más de 15 variables adicionales (por ejemplo, resolución de pantalla, número de núcleos del procesador, peso), por lo que incorporar más dimensiones **podría** mejorar la separación de los clusters, aunque también podría introducir el problema conocido como "maldición de la dimensionalidad", donde las distancias entre puntos pierden capacidad discriminativa a medida que aumentan las dimensiones. (*¿Qué Es La Reducción De La Dimensionalidad?*, n.d.) Otra alternativa sería probar un algoritmo de clustering distinto (como K-means o clustering jerárquico) para comparar si el problema persiste independientemente del método utilizado.

Finalmente, el aprendizaje no supervisado depende únicamente de patrones intrínsecos en los datos, sin retroalimentación externa. Esto lo hace considerablemente más sensible tanto a la selección de variables como a la configuración de hiperparámetros, y no garantiza que la estructura descubierta tenga relación con ninguna variable de interés en particular, como se demostró en este caso.

6. Referencias

DBSCAN — scikit-learn 1.9.0 documentation. (n.d.). Scikit-learn. Retrieved

September 9, 2026, from

<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>

Mobile Price Classification. (n.d.). Kaggle. Retrieved September 1, 2026, from

<https://www.kaggle.com/datasets/iabhishekoofficial/mobile-price-classification>

¿Qué es la reducción de la dimensionalidad? (n.d.). IBM. Retrieved September 9,

2026, from <https://www.ibm.com/mx-es/think/topics/dimensionality-reduction>